Artificial Intelligence Department – Candidate Assignment

Project Overview

Building an Image-Based Place Recognition & Retrieval System

Imagine a mobile robot, a delivery fleet, or a security team trying to determine **where a photo was taken** by comparing it against a reference gallery. In this assignment, you will build a lightweight place recognition system that **extracts image embeddings**, performs **similarity search**, and returns the most likely matching location(s).

Project Workflow

1) Dataset Curation & Safe Splitting

- An example of a labeled dataset containing images from multiple places/scenes will be provided. Any dataset/sample structure shared as an attachment or example is illustrative only. You should adapt the structure, add missing fields, and fix inconsistencies as needed—as long as the final pipeline remains reproducible and well-documented.
- Create a manifest (e.g., `train/val/test` or `gallery/query`) and ensure **no data leakage** occurs due to near-duplicates or repeated frames. If similar images exist, they **must not** appear on both sides of the split.
- **Do not use metadata** (e.g., file names, EXIF GPS tags, timestamps) for recognition. Only pixel content is allowed.
- The pipeline must be robust to real-world issues such as **corrupted images, grayscale images, and very small resolutions** (log and skip if needed).

2) Embedding Extraction

- Use a pretrained vision backbone to compute fixed-size embeddings (e.g., ResNet feature vector or an open embedding model).
- Use **batched inference** and apply **L2 normalization** before similarity search.
- Implement caching so repeated runs do not recompute embeddings unnecessarily (e.g., `.npy` shards or a structured cache directory).

3) Index Building & Similarity Search

- Build a gallery index using **cosine similarity** (a NumPy baseline is sufficient).
- Optional: implement approximate nearest neighbor search (FAISS is allowed but not required).

- Provide a CLI that retrieves **Top-K matches** for a given query image or a query folder.

4) Open-Set Handling (Unknown Location)

- Some query images may not have a matching location in the gallery.
- Implement a confidence strategy (e.g., max-similarity threshold, or margin between Top-1 and Top-2) and return **UNKNOWN** when appropriate.
- Clearly report the chosen strategy and explain how it affects results.

5) Evaluation & Reporting

- Compute **Recall@K** and **mean Average Precision (mAP)**.
    - Note: some queries may have multiple valid matches; ensure your implementation handles this correctly.
- Optional: include a lightweight performance benchmark (e.g., latency/throughput, RAM/VRAM usage).
- Include **at least two unit tests** validating your metric calculations (including a *multi-positive query* case).
- Analyze failure modes (e.g., day vs night, seasonal changes, viewpoint changes) and propose improvements.

Key Goals

- **Accurate retrieval:** achieve strong Recall@1/5/10 on the provided benchmark split.
- **Correct evaluation:** implement metrics carefully and validate them with tests.
- **Robustness:** handle duplicates, corrupted files, grayscale inputs, resolution changes, and domain shift.
- **High-quality software:** clean, modular code with documentation and a usable CLI.

Bonus (Optional)

- **Hard negative mining:** select hard negatives based on embedding similarity and show measurable improvement.
- **Faster embedding extraction:** use multiprocessing or asynchronous loading to speed up preprocessing/inference.
- **Performance notes:** provide a short benchmark (latency/throughput) for extraction and search.

- Optional: export the backbone to **ONNX** and run inference via **ONNX Runtime**. Or you can compare ONNX Runtime and Tensor Runtime if you want.

Project Requirements & Tools

Dataset and Language

- The dataset will be provided by the company.
- The project must be implemented in **Python 3.10+** (Python 3.11 is recommended).

Recommended Libraries

- **PyTorch** for model inference (training is not required).
- **OpenCV** for image I/O and preprocessing.
- **NumPy** for similarity search; optional **FAISS** for ANN indexing.
- **pytest** for unit tests (required).

Engineering Expectations

- Use a clear repository structure (e.g., `src/`, `tests/`, `scripts/`).
- Provide a reproducible setup (`requirements.txt` or `pyproject.toml`).
- If possible, include lint/format tooling (e.g., `ruff`, `black`).

Project Delivery Guidelines

- Upload your project to GitHub and share the repository link.
- Include a README with:
    - Installation steps
    - How to build the index
    - How to run search
    - How to run evaluation
- Provide the following scripts (names may match):
    - `extract_embeddings.py`
    - `build_index.py`
    - `search.py`
    - `evaluate.py`
- Include a short report (1–2 pages) summarizing:
    - Results (Recall@1/5/10, mAP)
    - Key observations
    - Suggested improvements

Time Expectation & Submission

This assignment is designed to be completed in approximately **3–4 days**.
Submission deadline: 09.02.2026 – 13:00

Please prioritize **correctness, clarity, and engineering quality** over heavy model training. We do not expect the fastest or the most accurate model, we expect to see how you think, build and develop.

Email submission:

- **To:** Mert Yüksel – AI Team Lead (myuksel@etetechnology.com)
- **CC:** Binnaz Yüceoğlu – Administrative Specialist (byuceoglu@etetechnology.com)
- **Subject:** “<Name Surname> AI Task Assignment”